MYSQL questions

Basic to conceptual questions

1. What is DBMS?
2. What is RDBMS?
3. What is the difference between DBMS and RDBMS?
4. What is a database?
5. What is a table, row, and column?
6. What is SQL? Why do we use it?
7. What are the advantages of MySQL?
8. What is the difference between SQL and MySQL?
9. What is a schema in MySQL?
10. What is the difference between structured and unstructured data?

Topic 2: SQL Basics & Commands

1. What are the different types of SQL commands?
2. Explain DDL, DML, DQL, DCL, and TCL with examples.
3. What is the difference between DELETE, DROP, and TRUNCATE?
4. What is the difference between INSERT and UPDATE?
5. What is the difference between WHERE and HAVING?
6. What is the difference between DISTINCT and UNIQUE?
7. What is the difference between CHAR and VARCHAR?
8. What is the difference between NULL, 0, and an empty string?
9. What is the purpose of the ORDER BY clause?
10. What is the difference between GROUP BY and ORDER BY?

Topic 3: Keys & Constraints

1. What is a Primary Key?
2. What is a Foreign Key?
3. What is the difference between Primary Key and Unique Key?
4. Can a table have multiple Primary Keys?
5. Can a Foreign Key contain NULL values?
6. What is a Composite Key?
7. What is a Candidate Key?
8. What is a Super Key?
9. What are SQL constraints?
10. Explain NOT NULL, UNIQUE, CHECK, DEFAULT, PRIMARY KEY, and FOREIGN KEY.

Topic 4: Operators & Clauses

1. What is the difference between IN and BETWEEN?
2. What is the difference between LIKE and REGEXP?
3. Explain the use of AND, OR, and NOT operators.
4. What are comparison operators in SQL?
5. What is the difference between ANY and ALL?
6. What is the use of the LIMIT clause?
7. What is the difference between IS NULL and = NULL?
8. What is the use of the CASE statement?
9. What is the difference between UNION and UNION ALL?
10. What is the difference between EXISTS and IN?

Topic 5: SQL Joins

1. What is a JOIN in SQL?

2. Why do we use JOINs?
3. Explain INNER JOIN.
4. Explain LEFT JOIN.
5. Explain RIGHT JOIN.
6. What is a FULL OUTER JOIN? Does MySQL support it directly?
7. What is a CROSS JOIN?
8. What is a SELF JOIN?
9. What is the difference between INNER JOIN and LEFT JOIN?
10. Can we join more than two tables?

Topic 6: SQL Functions & Grouping

1. What are aggregate functions?
2. Explain COUNT(), SUM(), AVG(), MIN(), and MAX().
3. What is the difference between COUNT(*) and COUNT(column_name)?
4. What is the difference between aggregate and scalar functions?
5. What are string functions in MySQL?
6. Explain CONCAT(), LENGTH(), UPPER(), LOWER(), and SUBSTRING().
7. What are date and time functions?
8. What is the purpose of GROUP BY?
9. What is the purpose of HAVING?
10. Can we use aggregate functions inside WHERE?

Topic 7: Subqueries

1. What is a subquery?
2. What are the different types of subqueries?
3. What is the difference between a subquery and a JOIN?

4. What is a correlated subquery?
5. What is the difference between correlated and non-correlated subqueries?
6. Can a subquery return multiple rows?
7. What is the difference between IN and EXISTS?
8. Write a query to find the second-highest salary using a subquery.

Topic 8: Normalization

1. What is normalization?
2. Why do we need normalization?
3. What are database anomalies?
4. Explain insertion, deletion, and update anomalies.
5. What is 1NF?
6. What is 2NF?
7. What is 3NF?
8. What is the difference between 2NF and 3NF?

Topic 9: Indexes & Views

1. What is an index in MySQL?
2. Why do we use indexes?
3. What are the advantages and disadvantages of indexes?
4. What is the difference between clustered and non-clustered indexes?
5. What is a composite index?
6. What is a view in MySQL?
7. What is the difference between a table and a view?
8. Can we update data through a view?

Topic 10: Transactions & ACID Properties

1. What is a transaction?
2. What are ACID properties?
3. Explain Atomicity, Consistency, Isolation, and Durability.
4. What is COMMIT?
5. What is ROLLBACK?
6. What is SAVEPOINT?
7. What is autocommit in MySQL?

Topic 11: Advanced SQL

1. What is a Stored Procedure?
2. What is a Function in MySQL?
3. What is the difference between Stored Procedure and Function?
4. What is a Trigger?
5. What are the different types of Triggers?
6. What is a Cursor?
7. What is a Window Function?
8. What is the difference between RANK(), DENSE_RANK(), and ROW_NUMBER()?

Topic 12: Practical SQL Query Questions

Assume you have the following tables:

Employee

emp_id	name	salary	dept_id
1	Rahul	30000	1
2	Aman	45000	2
3	Priya	50000	1
4	Mohit	25000	3

emp_id	name	salary	dept_id
5	Neha	45000	2

Department

dept_id	dept_name
1	IT
2	HR
3	Sales

Practice these:

1. Write a query to display all employees.
2. Find employees whose salary is greater than 30000.
3. Find the highest salary.
4. Find the second-highest salary.
5. Find the employee who receives the highest salary.
6. Count the total number of employees.
7. Find the average salary of employees.
8. Display department-wise average salary.
9. Find departments having more than one employee.
10. Find employees whose salary is equal to 45000.
11. Display employee names with their department names.
12. Find the highest salary in each department.
13. Find employees who are not assigned to any department.
14. Find duplicate salaries.
15. Find the third-highest distinct salary.

Database & MySQL: Fundamental Interview Questions

1. What is DBMS?

A **DBMS (Database Management System)** is software that serves as an interface between the user, the database, and application programs. It allows users to create, read, update, and manage data in an organized manner while ensuring data security, integrity, and concurrency control.

- *Examples:* MySQL, PostgreSQL, SQLite, Oracle.

---

2. What is RDBMS?

An **RDBMS (Relational Database Management System)** is an advanced type of DBMS based on the relational model. Data is stored in tabular form—consisting of rows and columns—and tables can be linked (related) to one another using keys (such as Primary and Foreign keys) to ensure data integrity and minimize redundancy.

---

3. What is the difference between DBMS and RDBMS?

Feature	DBMS	RDBMS
Data Storage	Stores data as files or in hierarchical/network formats.	Stores data in a tabular format (tables with rows and columns).
Relationships	Does not typically support relationships between data tables.	Supports explicit relationships between tables using keys (e.g., Foreign Keys).
Data Redundancy	Higher chance of data redundancy and duplication.	Minimized redundancy due to normalization rules.
Scalability	Generally suited for smaller amounts of data.	Highly scalable and handles complex, large-scale datasets efficiently.
Examples	XML, JSON files, Microsoft Access (classic file-based).	MySQL, PostgreSQL, Oracle, SQL Server.

---

4. What is a database?

A **database** is an organized, electronic collection of structured data or information stored in a computer system, designed to allow fast retrieval, manipulation, and secure storage by multiple users simultaneously.

---

5. What is a table, row, and column?

- **Table (Relation):** A structured collection of data organized into vertical columns and horizontal rows.
- **Row (Record / Tuple):** A single, horizontal entry in a table representing a unique record or instance of an entity (e.g., a specific employee's details).
- **Column (Field / Attribute):** A vertical entity in a table defining a specific attribute or data type for all rows (e.g., `employee_id`, `first_name`, `salary`).

---

6. What is SQL? Why do we use it?

SQL (Structured Query Language) is a standardized programming language specifically designed for managing, querying, and manipulating data stored in relational databases.

Why we use it:

- Provides a simple, English-like syntax to interact with databases.
- Performs CRUD operations (Create, Read, Update, Delete) efficiently.
- Handles complex data filtering, aggregations, joins, and database administration reliably.

---

7. What are the advantages of MySQL?

- **Open-Source & Free:** Freely available to use and modify under the GNU General Public License.
- **High Performance & Speed:** Optimized for heavy workloads, delivering fast query execution.
- **Scalability & Flexibility:** Capable of handling small personal projects as well as massive enterprise applications.

- **Strong Security:** Offers robust security protocols, user privilege management, and encrypted connections.
- **Cross-Platform:** Runs seamlessly on Windows, Linux, and macOS.
- **Extensive Ecosystem:** Integrates smoothly with popular programming languages like Python, Java, and PHP.

---

8. What is the difference between SQL and MySQL?

- **SQL** is the **language**—the set of rules and syntax used to communicate with a relational database.
- **MySQL** is the **software**—an actual Relational Database Management System (RDBMS) that processes SQL queries to manage data on disk or in memory.

---

9. What is a schema in MySQL?

In MySQL, a **schema** is a logical container or structural grouping for database objects, such as tables, views, stored procedures, triggers, and indexes. In practice, MySQL treats the `CREATE DATABASE` and `CREATE SCHEMA` commands as synonyms; both create a new database container.

---

10. What is the difference between structured and unstructured data?

- **Structured Data:** Highly organized, formatted, and predefined data models (such as tables with rows and columns in an RDBMS). Easily searchable and queryable using SQL.
- **Unstructured Data:** Data that lacks a predefined data model or organizational structure. Examples include text documents, emails, audio files, videos, and social media posts. Requires specialized tools (like NoSQL databases or AI processing) to analyze and store.

SQL Basics & Commands: Interview Q&A

1. What are the different types of SQL commands?

SQL commands are generally categorized into five major types based on their purpose:

1. **DDL** (Data Definition Language)
2. **DML** (Data Manipulation Language)
3. **DQL** (Data Query Language)
4. **DCL** (Data Control Language)
5. **TCL** (Transaction Control Language)

---

2. Explain DDL, DML, DQL, DCL, and TCL with examples.

- **DDL (Data Definition Language):** Used to define or modify the database structure or schema.
  - *Examples:* `CREATE`, `ALTER`, `DROP`, `TRUNCATE`, `RENAME`.
- **DML (Data Manipulation Language):** Used to modify, insert, or delete data within tables.
  - *Examples:* `INSERT`, `UPDATE`, `DELETE`.
- **DQL (Data Query Language):** Used to retrieve or query data from the database.
  - *Example:* `SELECT`.
- **DCL (Data Control Language):** Used to manage user permissions and access rights to the database.
  - *Examples:* `GRANT`, `REVOKE`.
- **TCL (Transaction Control Language):** Used to manage transactions in the database to maintain data consistency.
  - *Examples:* `COMMIT`, `ROLLBACK`, `SAVEPOINT`.

---

3. What is the difference between DELETE, DROP, and TRUNCATE?

Feature	DELETE	TRUNCATE	DROP
Command Type	DML (Data Manipulation)	DDL (Data Definition)	DDL (Data Definition)
Operation	Deletes specific rows (can use `WHERE` clause).	Removes all rows from a table without scanning row-by-row.	Deletes the entire table structure and all its data from the database.
Rollback	Can be rolled back if used inside a transaction.	Cannot be rolled back in MySQL (causes implicit commit).	Cannot be rolled back.
Triggers	Fires table triggers.	Does not fire triggers.	Does not fire triggers.
Auto-Increment	Keeps the current auto-increment value.	Resets the auto-increment counter to 1.	Destroys the table along with its sequence.

4. What is the difference between INSERT and UPDATE?

- **INSERT:** Used to add brand-new rows (records) into a table.
- **UPDATE:** Used to modify or change existing values in current rows based on specific conditions (`WHERE` clause).

5. What is the difference between WHERE and HAVING?

- **WHERE:** Filters rows *before* any grouping or aggregation takes place. It operates on individual columns/rows and cannot use aggregate functions (`SUM`, `COUNT`, etc.).
- **HAVING:** Filters groups *after* aggregation has occurred. It is always used in conjunction with `GROUP BY` and can contain aggregate functions.

6. What is the difference between DISTINCT and UNIQUE?

- **DISTINCT:** A keyword used within a `SELECT` query to remove duplicate rows from the final result set.
- **UNIQUE:** A column-level or table-level constraint applied during table creation to ensure that no two rows can have the same value in that specific column.

7. What is the difference between CHAR and VARCHAR?

- **CHAR (Character):** A **fixed-length** data type. If the inserted string is shorter than the defined length, MySQL pads it with trailing spaces to fill the full length. It is faster for fixed-size columns (like country codes or status flags).
- **VARCHAR (Variable Character):** A **variable-length** data type. It only stores the exact length of the characters entered plus a 1 or 2-byte prefix tracking length, making it much more storage-efficient for variable text.

---

8. What is the difference between NULL, 0, and an empty string?

- **NULL:** Represents the complete absence of a value, unknown data, or an unassigned field.
- **0:** A valid numeric value representing the number zero.
- **Empty String (`''`):** A string data type with a length of zero characters. It occupies memory space for a string, unlike `NULL` which takes no data value.

---

9. What is the purpose of the ORDER BY clause?

The **ORDER BY** clause is used to sort the result set of a query in either ascending (`ASC`, default) or descending (`DESC`) order based on one or more specified columns.

---

10. What is the difference between GROUP BY and ORDER BY?

- **GROUP BY:** Collects and aggregates data across multiple rows that share common values into summary groups, usually paired with aggregate functions like `COUNT()`, `SUM()`, or `AVG()`.
- **ORDER BY:** Merely sorts the final result set of rows into a specific sequence without grouping or combining rows together.

Keys & Constraints: Interview Q&A

1. What is a Primary Key?

A **Primary Key** is a constraint used to uniquely identify each row or record in a table.

- **Characteristics:** It must contain unique values and **cannot** contain `NULL` values.
- **Limit:** A table can have only **one** primary key (though that key can span multiple columns).

---

2. What is a Foreign Key?

A **Foreign Key** is a column (or set of columns) in one table that references the **Primary Key** of another table.

- **Purpose:** It establishes a link between the two tables and enforces **referential integrity**, ensuring that data cannot be inserted into the child table if it doesn't have a corresponding match in the parent table.

---

3. What is the difference between Primary Key and Unique Key?

Feature	Primary Key	Unique Key
Null Values	Does **not** allow any `NULL` values.	Allows **one** `NULL` value (in MySQL).
Number per Table	Only **one** primary key is allowed per table.	Multiple unique keys can be defined per table.
Clustered Index	Automatically creates a clustered index on the table.	Automatically creates a non-clustered index by default.

---

4. Can a table have multiple Primary Keys?

No, a table can only have **one** Primary Key constraint. However, that single Primary Key can consist of **multiple columns** combined together (which is known as a composite primary key).

---

5. What is a Composite Key?

A **Composite Key** is a type of primary key made by combining two or more columns in a table to uniquely identify a record when no single column alone is unique enough.

---

6. Can a Foreign Key contain NULL values?

Yes, a foreign key column can contain `NULL` values unless an explicit `NOT NULL` constraint is added. A `NULL` foreign key means the record does not have a relationship or association with any parent table record.

---

7. What is a Candidate Key?

A **Candidate Key** is any column or combination of columns that can uniquely identify a record in a table.

- A table can have multiple candidate keys.
- Out of all candidate keys, database designers choose **one** to act as the Primary Key; the remaining ones are called **alternate keys**.

---

8. What is a Super Key?

A **Super Key** is any combination of columns that can uniquely identify a row in a table.

- A super key can contain extra, unnecessary columns beyond what is required for uniqueness.
- *Note:* Every candidate key is a super key, but not every super key is a candidate key.

---

9. What are SQL constraints?

SQL constraints are rules and restrictions applied to columns or tables during creation to ensure data accuracy, reliability, and integrity. If an operation violates a constraint, the action is aborted by the database.

---

10. Explain NOT NULL, UNIQUE, CHECK, DEFAULT, PRIMARY KEY, and FOREIGN KEY.

- **NOT NULL:** Ensures that a column cannot store a `NULL` value (a value must always be provided).
- **UNIQUE:** Ensures that all values in a column are distinct from one another.

- **CHECK:** Ensures that all values in a column satisfy a specific logical condition or expression (e.g., `age >= 18`).
- **DEFAULT:** Automatically assigns a preset default value to a column if no value is explicitly provided during an `INSERT`.
- **PRIMARY KEY:** Uniquely identifies each record in a table by combining `NOT NULL` and `UNIQUE` properties.
- **FOREIGN KEY:** Maintains referential integrity by linking a column's values to the primary key of another table.

Advanced SQL & Database Management: Interview Q&A

Topic 4: Operators & Clauses

1. What is the difference between IN and BETWEEN?

- **IN:** Checks if a column's value matches any value within a specified list of discrete values (e.g., `department_id IN (10, 20, 30)`).
- **BETWEEN:** Checks if a column's value falls within a specific inclusive range (e.g., `salary BETWEEN 50000 AND 80000`).

---

2. What is the difference between LIKE and REGEXP?

- **LIKE:** Used for basic wildcard pattern matching using `%` (matches zero or more characters) and `_` (matches a single character).
- **REGEXP (or RLIKE):** Used for advanced, complex pattern matching using regular expressions (e.g., anchoring, character classes, repetitions).

---

3. Explain the use of AND, OR, and NOT operators.

- **AND:** Combines multiple conditions; returns true only if **all** conditions are met.
- **OR:** Combines multiple conditions; returns true if **at least one** condition is met.

- **NOT:** Negates or reverses the result of a boolean condition (e.g., `NOT LIKE`, `NOT IN`).

---

4. What are comparison operators in SQL?

Comparison operators are used to compare data values in a `WHERE` or `HAVING` clause.

- *Examples:* `=` (Equal), `<>` or `!=` (Not equal), `>` (Greater than), `<` (Less than), `>=` (Greater than or equal), `<=` (Less than or equal).

---

5. What is the difference between ANY and ALL?

- **ANY:** Returns true if the comparison is met for *at least one* value in a subquery result set.
- **ALL:** Returns true only if the comparison is met for *every* value in the subquery result set.

---

6. What is the use of the LIMIT clause?

The **LIMIT** clause is used to restrict the number of rows returned by a `SELECT` query. It is especially useful for pagination (often paired with `OFFSET`) and inspecting subsets of large datasets.

---

7. What is the difference between IS NULL and = NULL?

- **IS NULL:** The correct and proper way to check for missing or `NULL` values in SQL because `NULL` is an unknown state, not a concrete value.
- **= NULL:** Incorrect syntax that always evaluates to unknown/false because nothing can be mathematically equated to `NULL` using standard comparison operators.

---

8. What is the use of the CASE statement?

The **CASE** statement acts as conditional logic (similar to if-else statements in programming) inside SQL queries. It evaluates conditions sequentially and returns a specific result when the first matching condition is met.

---

9. What is the difference between UNION and UNION ALL?

- **UNION:** Combines the result sets of two or more `SELECT` queries and **removes duplicate rows** from the final output (which incurs a performance overhead for sorting/deduplication).
- **UNION ALL:** Combines result sets and **keeps all rows, including duplicates**, making it faster because it skips the deduplication step.

---

10. What is the difference between EXISTS and IN?

- **IN:** Evaluates values against a static list or the full result set of a subquery. It can be less efficient on very large datasets.
- **EXISTS:** Evaluates boolean existence; it stops processing as soon as it finds the first matching row in the subquery (short-circuit evaluation), making it highly efficient for correlated queries.

Topic 5: SQL Joins

1. What is a JOIN in SQL?

A **JOIN** is a clause used to combine rows from two or more tables based on a related logical column (relationship) between them.

---

2. Why do we use JOINs?

We use JOINs to retrieve normalized data spread across multiple separate tables into a single unified result set without violating database normalization rules.

---

3. Explain INNER JOIN.

An **INNER JOIN** returns only the records that have matching values in **both** tables participating in the join. Unmatched rows are excluded.

---

4. Explain LEFT JOIN.

A **LEFT JOIN** returns all records from the left table, and the matched records from the right table. If there is no match in the right table, the result contains `NULL`

values for the right table's columns.

---

5. Explain RIGHT JOIN.

A **RIGHT JOIN** returns all records from the right table, and the matched records from the left table. If there is no match in the left table, the result contains `NULL` values for the left table's columns.

---

6. What is a FULL OUTER JOIN? Does MySQL support it directly?

- **FULL OUTER JOIN:** Returns all records when there is a match in either the left or right table, filling with `NULL` where data is missing on either side.
- **MySQL Support:** MySQL **does not** support `FULL OUTER JOIN` syntax directly. It must be simulated by combining a `LEFT JOIN` and a `RIGHT JOIN` using `UNION`.

---

7. What is a CROSS JOIN?

A **CROSS JOIN** produces the Cartesian product of two tables. Every row from the first table is combined with every row from the second table, resulting in $Rows_A \times Rows_B$ total rows.

---

8. What is a SELF JOIN?

A **SELF JOIN** is a regular join where a table is joined with itself. This is commonly used for hierarchical data, such as finding an employee's manager within the same `Employees` table.

---

9. What is the difference between INNER JOIN and LEFT JOIN?

- **INNER JOIN:** Returns only matching rows from both tables.
- **LEFT JOIN:** Returns all rows from the left table, plus matched rows from the right table (with `NULL` for unmatched right rows).

---

10. Can we join more than two tables?

Yes, multiple `JOIN` clauses can be chained sequentially in a single query to connect three or more tables together.

Topic 6: SQL Functions & Grouping

1. What are aggregate functions?

Aggregate functions perform a calculation across a set of multiple rows and return a **single summary value** (e.g., total sum, average, count).

---

2. Explain COUNT(), SUM(), AVG(), MIN(), and MAX().

- **COUNT():** Returns the number of rows matching criteria.
- **SUM():** Calculates the total sum of a numeric column.
- **AVG():** Calculates the average value of a numeric column.
- **MIN():** Finds the smallest value in a column.
- **MAX():** Finds the largest value in a column.

---

3. What is the difference between COUNT(*) and COUNT(column_name)?

- **COUNT(*):** Counts **all** rows in the result set, including rows with `NULL` values and duplicate rows.
- **COUNT(column_name):** Counts only the rows where the specified column contains **non-NULL** values, ignoring `NULL` entries.

---

4. What is the difference between aggregate and scalar functions?

- **Aggregate Functions:** Operate on sets of multiple rows to return a single summarized value.
- **Scalar Functions:** Operate on a single row/value at a time and return a single modified value (e.g., `UPPER()`, `LENGTH()`).

---

5. What are string functions in MySQL?

Built-in functions used to manipulate, inspect, or format string/text data values in queries.

---

6. Explain CONCAT(), LENGTH(), UPPER(), LOWER(), and SUBSTRING().

- **CONCAT():** Joins two or more strings together.
- **LENGTH():** Returns the length of a string in bytes.
- **UPPER():** Converts all letters in a string to uppercase.
- **LOWER():** Converts all letters in a string to lowercase.
- **SUBSTRING():** Extracts a portion of a string starting from a specified position for a given length.

---

7. What are date and time functions?

Functions used to manipulate and retrieve date/time values (e.g., `NOW()`, `CURDATE()`, `DATEDIFF()`, `DATE_ADD()`).

---

8. What is the purpose of GROUP BY?

The **GROUP BY** clause groups rows that share common values into summary rows, allowing aggregate functions to be performed on each group independently.

---

9. What is the purpose of HAVING?

The **HAVING** clause is used to filter groups created by `GROUP BY` based on aggregate conditions (since the `WHERE` clause cannot accept aggregate functions).

---

10. Can we use aggregate functions inside WHERE?

No, aggregate functions cannot be used in the `WHERE` clause because `WHERE` filters individual rows *before* grouping and aggregation take place. Use `HAVING` instead.

Topic 7: Subqueries

1. What is a subquery?

A **subquery** (or inner query / nested query) is a query nested inside another SQL statement (such as `SELECT`, `INSERT`, `UPDATE`, or `DELETE`).

---

2. What are the different types of subqueries?

- **Scalar Subquery:** Returns a single row and single column.
- **Row Subquery:** Returns a single row with multiple columns.
- **Table Subquery:** Returns multiple rows and columns.
- **Correlated Subquery:** References columns from the outer query.

---

3. What is the difference between a subquery and a JOIN?

- **Subquery:** Nests a query inside another; often used when data from an inner calculation is needed to filter outer rows.
- **JOIN:** Combines columns and rows from multiple tables side-by-side based on a relational condition; generally more performant for large-scale table linking.

---

4. What is a correlated subquery?

A **correlated subquery** is a subquery that depends on the outer query for its values; it executes repeatedly—once for each row evaluated by the outer query.

---

5. What is the difference between correlated and non-correlated subqueries?

- **Non-Correlated Subquery:** Can be executed independently once, and its result is passed to the outer query.
- **Correlated Subquery:** Evaluates dynamically row-by-row, depending directly on data from the outer query.

---

6. Can a subquery return multiple rows?

Yes, when used with operators like `IN`, `ANY`, or `ALL`, subqueries can return multiple rows.

---

7. What is the difference between IN and EXISTS?

- **IN:** Compares a column against a complete list of values generated by the subquery.
- **EXISTS:** Checks for the presence of any matching rows returned by the subquery using short-circuit evaluation.

---

8. Write a query to find the second-highest salary using a subquery.

```sql
SELECT MAX(salary) AS second_highest_salary
FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);
```

Topic 8: Normalization

1. What is normalization?

Normalization is the systematic process of organizing data in a database to reduce data redundancy and eliminate undesirable anomalies (insertion, update, deletion).

---

2. Why do we need normalization?

We need normalization to save storage space, ensure data consistency and integrity, and prevent anomalies during DML operations.

---

3. What are database anomalies?

Database anomalies are flaws or irregularities that occur in unnormalized databases when performing basic operations (`INSERT`, `UPDATE`, `DELETE`), leading to data corruption or inconsistency.

---

4. Explain insertion, deletion, and update anomalies.

- **Insertion Anomaly:** Inability to add certain data without adding extraneous, unrelated dummy data.

- **Deletion Anomaly:** Accidental loss of valid secondary data when deleting a primary record.
- **Update Anomaly:** Having to update duplicated data in multiple places, leading to partial updates and inconsistencies.

---

5. What is 1NF (First Normal Form)?

- Every column must contain atomic (indivisible) values.
- Each column must contain values of a single data type.
- Each row must be unique (having a primary key).

---

6. What is 2NF (Second Normal Form)?

- Must already be in **1NF**.
- All non-key attributes must be fully functionally dependent on the **entire primary key** (no partial dependencies allowed, which applies primarily to composite keys).

---

7. What is 3NF (Third Normal Form)?

- Must already be in **2NF**.
- There must be **no transitive dependencies** (i.e., non-key columns should not depend on other non-key columns).

---

8. What is the difference between 2NF and 3NF?

- **2NF:** Eliminates **partial dependencies** (where a non-key column depends on only part of a composite primary key).
- **3NF:** Eliminates **transitive dependencies** (where a non-key column depends on another non-key column).

Topic 9: Indexes & Views

1. What is an index in MySQL?

An **index** is a special data structure (typically a B-Tree) created on database table columns to speed up data retrieval operations.

2. Why do we use indexes?

We use indexes to dramatically reduce query execution time so the database engine doesn't have to perform slow full-table scans.

3. What are the advantages and disadvantages of indexes?

- **Advantages:** Blazing-fast `SELECT` query and search speeds.
- **Disadvantages:** Slower `INSERT`, `UPDATE`, and `DELETE` performance because indexes must be updated; consumes extra disk storage space.

4. What is the difference between clustered and non-clustered indexes?

- **Clustered Index:** Sorts and stores the data rows physically in the table based on the index key. A table can have only **one** clustered index (usually the Primary Key).
- **Non-Clustered Index:** Creates a separate structure containing index pointers that reference the underlying data rows. A table can have **multiple** non-clustered indexes.

5. What is a composite index?

A **composite index** is an index created on two or more columns combined together, following the Left-Most Prefix rule for query optimization.

6. What is a view in MySQL?

A **view** is a virtual table whose contents are defined by a stored SQL query. It does not store physical data itself (unless it is a materialized view, though standard MySQL uses virtual views).

7. What is the difference between a table and a view?

- **Table:** Stores actual physical data on disk.
- **View:** Does not store data physically; it acts as a saved window or query projection onto underlying tables.

---

8. Can we update data through a view?

Yes, but only under certain conditions. Updatable views must maintain a direct one-to-one mapping with underlying base tables without aggregate functions, `GROUP BY`, `DISTINCT`, or `UNION`.

Topic 10: Transactions & ACID Properties

1. What is a transaction?

A **transaction** is a sequence of one or more SQL operations executed as a single logical unit of work.

---

2. What are ACID properties?

ACID represents the four key properties that guarantee reliable database transactions: **Atomicity, Consistency, Isolation, and Durability**.

---

3. Explain Atomicity, Consistency, Isolation, and Durability.

- **Atomicity ("All or Nothing"):** Either all operations in a transaction succeed, or none take effect.
- **Consistency:** The database transitions from one valid state to another, maintaining all constraints and rules.
- **Isolation:** Concurrent transactions execute independently without interfering with one another.
- **Durability:** Once a transaction is committed, its changes are permanent, even in the event of a system crash.

---

4. What is COMMIT?

COMMIT is a transaction control command that saves all changes made during the current transaction permanently to the database.

5. What is ROLLBACK?

ROLLBACK undoes all changes made during the current transaction, reverting the database to its state before the transaction began.

6. What is SAVEPOINT?

A **SAVEPOINT** sets a named rollback marker within a transaction, allowing you to roll back to a specific intermediate point rather than aborting the entire transaction.

7. What is autocommit in MySQL?

By default, MySQL runs in **autocommit mode**, meaning every individual SQL statement is treated as an independent transaction and committed immediately.

Topic 11: Advanced SQL

1. What is a Stored Procedure?

A **Stored Procedure** is a precompiled collection of SQL statements and procedural logic (loops, variables) stored in the database server that can be executed on demand.

2. What is a Function in MySQL?

A **Function** is a database routine that accepts parameters, performs calculations or logic, and **must return a single value**, allowing it to be used directly inside SQL queries.

3. What is the difference between Stored Procedure and Function?

- **Procedure:** Can return multiple output parameters, perform DML operations, and cannot be called directly inside a `SELECT` statement.

- **Function:** Must return a single value, cannot perform DML operations, and can be invoked inline within SQL queries.

---

4. What is a Trigger?

A **Trigger** is a database object that automatically executes (fires) in response to specific DML events (`INSERT`, `UPDATE`, `DELETE`) occurring on a table.

---

5. What are the different types of Triggers?

- **Timing:** `BEFORE` or `AFTER`
- **Event:** `INSERT`, `UPDATE`, or `DELETE`
- *(Combined into types like `BEFORE INSERT`, `AFTER UPDATE`, etc.)*

---

6. What is a Cursor?

A **Cursor** is a database mechanism used to iterate through result sets row-by-row inside stored procedures or programmatic blocks when set-based operations are insufficient.

---

7. What is a Window Function?

A **Window Function** performs calculations across a set of table rows (a "window") that are somehow related to the current row, without collapsing rows into groups like `GROUP BY` does.

---

8. What is the difference between RANK(), DENSE_RANK(), and ROW_NUMBER()?

- **ROW_NUMBER():** Assigns a unique sequential integer to each row, even if values are identical.
- **RANK():** Assigns ranking numbers with gaps left in the sequence when there are ties (e.g., 1, 2, 2, 4).
- **DENSE_RANK():** Assigns ranking numbers without gaps when there are ties (e.g., 1, 2, 2, 3).